ORML Assignment 2 [50 points]: (Deep) Reinforcement Learning

Guidelines

- The deadline is Dec 17 at 12:00 (noon). Late submissions will not be accepted;
- Questions can be asked during the dedicated lab on Dec 05;
- This assignment has AI-index 3. Please refer to the "Rules on use of AI.pdf" available on Canvas to see what is allowed and not allowed. This also means that you must submit a technology statement with your deliverables;
- Grading is based on the provided marking scheme;
- You may use lecture and lab materials for inspiration for your code, graphs, and answers;
- There is no strict limit on the report length, but it should be consistent with the content. In particular, using generative AI as a filler will be penalized;
- Manage your time carefully: do not start the assignment at the last minute, as your agents will need time to train;
- Please do not use Jupyter Notebook.

On January 6, teachers will meet with a random selection of the students who submitted this assignment. In these meetings, you will be asked some questions about your own work, to assess whether indeed this is your own work that you fully understand. For this, **everyone** needs to be present in room K1201 on January 6, from 14:00 to 15:30.

Deliverables

Your Canvas submission should contain three elements:

1. A zip file called "Part1" containing all deliverables requested in Part 1;
2. A zip file called "Part2" containing all deliverables requested in Part 2;
3. The following technology statement **if you used an AI tool** (replace the text in capital letters with the requested information): *During the preparation of this work, I used [NAME TOOL / SERVICE / VERSION OF AI TOOL] in order to [REASON]. The following parts of the assignment were affected/generated by AI tool usage: [INTRODUCTION / METHODS / xxx, DISCUSSION]. After using this tool/service, [NAME STUDENT] evaluated the validity of the tool's outputs, including the sources that generative AI tools have used, and edited the content as needed. As a consequence, [NAME STUDENT] takes full responsibility for the content of their work.*
   OR
   The following technology statement **if you did not use any AI tool** (replace the text in capital letters with the requested information): *During the preparation of this work, I, [NAME STUDENT], did not use any AI tool.*

Good luck with the assignment!

Part 1 [25 points]: Designing an RL agent to solve the influencer game

Problem description

We study the influencer game, in which one simulates interactions between an influencer (the RL agent) and an influencee. Throughout the game, three values are associated with the influencee: a trust coefficient t and a value v, both of which evolve during the game, along with a value threshold V that remains fixed. While the influencee's value v is known to the agent, its trust coefficient t and value threshold V are not. The objective of the agent is to maximize the value gained from the influencee while gradually increasing its trust. To do so, the agent can perform four types of interactions:

- Initial contact, with fixed cost $c_1 = 100$ for the agent. After this interaction, the influencee invests an initial amount i_1 (i.e., $v \leftarrow i_1$), and is assigned an initial trust coefficient t_1 (i.e., $t \leftarrow t_1$) and a value threshold V. i_1 is randomly generated from a discrete uniform distribution $[0, 100]$, t_1 is randomly generated from a discrete uniform distribution $[0, 80]$, and V is randomly generated from a discrete uniform distribution $[50\,000, 400\,000]$. An initial contact can only be established once and is required to unlock the three other actions.
- Trust building, with fixed cost $c_2 = 100$ for the agent. There are two outcomes for this interaction:
  - The influencee loses trust in the agent, which occurs with probability $1 - \sqrt{\frac{t}{100}}$; this results in the end of the game, with the agent gaining 0 value from the influencee.
  - The influencee gains trust in the agent and increases its value, which occurs with probability $\sqrt{\frac{t}{100}}$; this results in the influencee investing an additional amount i_2 (i.e., $v \leftarrow v + i_2$), and increasing its trust coefficient by $\lfloor \frac{100-t}{4} \rfloor$ (i.e., $t \leftarrow t + \lfloor \frac{100-t}{4} \rfloor$). i_2 is also randomly generated from a discrete uniform distribution $[0, 100]$.

  Trust building can be repeated as many times as the agent wishes.
- Value building, with fixed cost $c_3 = 500$ for the agent. There are two outcomes for this interaction:
  - The influencee loses trust in the agent, which occurs with probability $1 - (\frac{t}{100})^2$; this results in the end of the game, with the agent gaining 0 value from the influencee.
  - The influencee gains trust in the agent and increases its value, which occurs with probability $(\frac{t}{100})^2$; this results in the influencee doubling its investment (i.e., $v \leftarrow v + v$), and increasing its trust coefficient by t_3 (i.e., $t \leftarrow t + t_3$). t_3 is randomly generated from a discrete uniform distribution $[0, 3]$.

  Value building can be repeated as many times as the agent wishes.
- Harvesting, with no cost for the agent; this results in the end of the game, with the agent gaining v value from the influencee.

If, at any point, the influencee's trust coefficient t is greater than or equal to 100, or if its value v is greater than or equal to its value threshold V, this also results in the end of the game, with the agent gaining 0 value from the influencee. The score of the game is determined either after harvesting or when the influencee loses trust, and consists of the value gained from the influencee minus the sum of all costs incurred by the agent to perform its interactions. For example:

- The agent chooses to make an initial contact, and after two trust building actions and three value building actions, the influencee loses trust in the agent. The total profit is $0 - 100 - (2 \times 100) - (3 \times 500) = -1800$.
- The agent chooses to make an initial contact, and after five trust building actions and four value building actions bringing the influencee's value v to 4350, the agent decides to harvest. The total profit is $4350 - 100 - (5 \times 100) - (4 \times 500) = 1750$.
- The agent chooses to not make any initial contact. The total profit is 0.

Tasks

Develop a Q-learning approach in Python aimed at maximizing profit for the agent in the influencer game. The work is divided into three components:

1. **Design**: the phase in which the states, actions, and rewards are identified. In the report, you must clearly describe and motivate the chosen states, actions, and rewards.
2. **Parameter search**: the phase in which the Q-learning parameters are determined. In the report, you must clearly explain how each parameter value was chosen. A large proportion of these explanations must be supported by computational experiments evaluating different parameter sets.
3. **Tests**: the phase in which the resulting approach is evaluated. In the report, you must provide convincing evidence demonstrating the quality of your approach. In particular, you must compare your approach with simple baseline strategies, which you must also develop. You must also describe the policy learned by the trained agent and explain why this policy makes sense. To assess performance, you must also submit a self-contained Python file named `performance.py` that calls your trained agent on a set of 100 000 randomly generated instances, which your file should also generate as per the problem description. This file should run in a few seconds and must print a single vector $[x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8]$, where:
   - x_1 is the average return of the agent over the 100 000 episodes;
   - x_2 is the average value of the influencee that the agent harvested;
   - x_3 is the average cost the agent spent per influencee over the 100 000 episodes;
   - x_4 is the number of times an influencee left after a trust-building operation;
   - x_5 is the number of times an influencee left after a value-building operation;
   - x_6 is the number of times an influencee left because its value exceeded the value threshold;
   - x_7 is the number of times an influencee left because its trust coefficient exceeded 100;
   - x_8 is the number of times an influencee was harvested.

   Note that this file will be run several times with different random seeds.

Deliverables

Zip file "Part1" should contain:

- All Python files necessary to produce the outputs displayed in your report.
- `performance.py`, the file that we will use to assess the performance of your approach.
- A report containing the information requested in the exercise.

Grading

Your grade will be determined using the following marking scheme:

Aspect	**Min**	**Max**	**Criteria**
Design	0	5	- The motivation behind the chosen states, actions, and rewards is clearly explained in the report. - The reasoning is coherent, and there are no apparent flaws in the decisions made.
Parameter search	0	5	- The motivation behind the chosen parameters is clearly explained in the report. - The reasoning is coherent, and there are no apparent flaws in the decisions made.
Tests	0	5	- The evidence showing that the approach outperforms simple baseline strategies is convincing. - The description of the policy found by the trained agent is clear and the policy makes sense.
Effort & Creativity	0	5	- Visible effort was invested in the design of the approach, with choices that are both innovative and meaningful. - Visible effort was invested in the report, with the appropriate use of graphs, figures, and tables when relevant.
Performance & Correctness	0	5	- Each phase (design, parameter search, tests) is correctly implemented. - The code and the report are consistent. - The approach performs well on the 100 000 benchmark instances tested when rerunning `performance.py`.

Part 2 [25 points]: Designing an RL and a deep RL heuristic for the 0-1 quadratic knapsack problem

Problem description

We study the 0-1 quadratic knapsack problem, which can be defined as follows: given a knapsack with capacity c, a set of n items, each with a weight w_i and a profit p_i ($i = 1, \ldots, n$), and a set of combined profits that specifies, for each pair of items ($i = 1, \ldots, n-1,\ j = i+1, \ldots, n$), the additional profit p_{ij} earned if both items i and j are selected in the knapsack, the 0-1 Quadratic Knapsack Problem (0-1 QKP) consists of finding a subset of items with the maximum total profit that fits within the knapsack. In the following, we assume that all weights, profits, and combined profits are non-negative integers. An introduction to the problem is provided in [1].

Instance generator

An instance generator for the 0-1 QKP called `instanceGenerator.py` is available on Canvas. It uses one parameter, `n`, representing the number of items. You are free to modify this Python file to generate as many instances as you wish, or to integrate the generator into one of your own files.

Exercise 1

[13 points] When all combined profits are non-negative, the 0-1 QKP can be modeled as follows:

$$\max \quad \sum_{i=1}^{n} p_i x_i + \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} p_{ij} y_{ij} \tag{1}$$

$$\text{s.t.} \quad \sum_{i=1}^{n} w_i x_i \leq c, \tag{2}$$

$$y_{ij} \leq x_i \qquad i = 1, \ldots, n-1,\ j = i+1, \ldots, n, \tag{3}$$

$$y_{ij} \leq x_j \qquad i = 1, \ldots, n-1,\ j = i+1, \ldots, n, \tag{4}$$

$$x_i \in \{0, 1\} \qquad i = 1, \ldots, n, \tag{5}$$

$$y_{ij} \in \{0, 1\} \qquad i = 1, \ldots, n-1,\ j = i+1, \ldots, n. \tag{6}$$

Here, the variable x_i takes the value 1 if item i is included in the knapsack, and the variable y_{ij} takes the value 1 if both items i and j are included. The objective function (1) maximizes the total profit. Constraint (2) ensures that the knapsack capacity is not exceeded. Constraints (3) and (4) ensure that y_{ij} can take the value 1 only if both x_i and x_j also take the value 1.

We aim to develop a hyper-heuristic approach for the 0-1 QKP in which items are iteratively added to the knapsack using a greedy strategy until a specified criterion is met, after which model (1)-(6) is solved on the reduced instance using an ILP solver. Intuitively, if only the greedy heuristic is used (i.e., if the specified criterion is never met), the resulting approach should be very fast, but the solution quality may be poor. Conversely, if model (1)-(6) is solved on the complete instance (i.e., if the specified criterion is met immediately), the resulting approach would return the optimal solution, but the computation time would be very long. If a time limit is imposed, solving model (1)-(6) on the complete instance may still return a feasible solution, but its quality is not necessarily good.

After developing an effective greedy approach for the 0-1 QKP and identifying a suitable form of stopping criterion for the greedy phase (e.g., the number of items added to the knapsack), implement a Q-learning algorithm in Python to determine the best threshold for this criterion. The hyper-heuristic must be fine-tuned for instances with `n` = 200, taking into account that the time limit for solving model (1)-(6) is set to 15 seconds. The work is divided into two components:

1. **Design and parameter setting:** the phase in which the states, actions, and rewards are identified, and a choice for the Q-learning parameters is made. In the report, you must clearly describe and motivate the chosen states, actions, rewards, and parameters. Note that, unlike **Part 1**, a parameter search in which different parameter sets are evaluated is not required for this exercise.

2. **Tests:** the phase in which the resulting approach is evaluated. In the report, you must present experimental results comparing your approach with (i) a fully greedy strategy and (ii) model (1)-(6) solved on the complete instance with a 15-second time limit. To assess performance, you must also submit a self-contained Python file named `performanceEx1.py` that runs your trained hyper-heuristic on a set of 20 instances called `instance0.txt`, ..., `instance19.txt` stored in the folder `InstancesEx1`, which we will generate with `instanceGenerator.py` using a different random seed. This file must produce an output file `results1.txt` containing one row per instance, where each row should contain a single number representing the solution value obtained by the hyper-heuristic for that instance (see the file `performance_example1.py` for an example).

Exercise 2

[12 points] We aim to develop a deep Q-learning heuristic for the 0-1 QKP in which an agent iteratively decides which item to insert next into the knapsack. Unlike **Exercise 1**, the agent here directly selects one item (from a set) to add, rather than choosing a heuristic move. The work is divided into three components:

1. **Design:** the phase in which the states, actions, and rewards are identified. In the report, you must clearly describe and motivate the chosen states, actions, and rewards.

2. **Training the deep Q network:** the phase in which the deep Q network is trained. Unlike the other exercises, here we value the fact that the agent has learned something more than the final performance (although the latter will still be checked). You must include in the report meaningful evidence that the agent is learning throughout the training process. You must also save the trained deep Q network so that it can be loaded and used in a testing file.

3. **Tests:** To assess performance, you must also submit a self-contained Python file named `performanceEx2.py` that uses your trained model to run the deep Q-learning heuristic on a set of 20 instances called `instance0.txt`, ..., `instance19.txt` stored in the folder `InstancesEx2`, which we will generate with `instanceGenerator.py` using a different random seed. This file must produce an output file `results2.txt` containing one row per instance, where each row should contain a single number representing the solution value obtained by the deep Q-learning heuristic for that instance (see the file `performance_example2.py` for an example).

Deliverables

Zip file "Part2" should contain:

- All Python files necessary to produce the outputs displayed in your report.

- `performance1.py` and `performance2.py`, the files that we will use to assess the performance of your two approaches.
- A report containing the information requested in the two exercises.

Grading

Your grade will be determined using the following marking scheme for Exercise 1:

Aspect	**Min**	**Max**	**Criteria**
Design	0	4	- The motivation behind the chosen states, actions, rewards, and parameters is clearly explained in the report. - The reasoning is coherent, and there are no apparent flaws in the decisions made.
Tests	0	2	- The evidence showing that the approach outperforms both a fully greedy strategy and model (1)-(6) solved on the complete instance with a 15-second time limit is convincing.
Effort & Creativity	0	3	- Visible effort was invested in the hyper-heuristic design, with choices that are both innovative and meaningful. - Visible effort was invested in the report, with the appropriate use of graphs, figures, and tables when relevant.
Performance & Correctness	0	4	- Each phase (design & parameter setting and tests) is correctly implemented. - The code and the report are consistent. - The approach performs well on the 20 benchmark instances.

Your grade will be determined using the following marking scheme for Exercise 2:

Aspect	**Min**	**Max**	**Criteria**
Design	0	4	- The motivation behind the chosen states, actions, and rewards is clearly explained in the report. - The reasoning is coherent, and there are no apparent flaws in the decisions made.
Training	0	3	- The evidence showing that the agent has learned something throughout the training is convincing.
Effort & Creativity	0	3	- Visible effort was invested in the deep Q-learning heuristic design, with choices that are both innovative and meaningful. - Visible effort was invested in the report, with the appropriate use of graphs, figures, and tables when relevant.
Performance & Correctness	0	2	- Each phase (design and training of the deep Q network) is correctly implemented. - The code and the report are consistent. - The approach performs well on the 20 benchmark instances.

References

[1] Alain Billionnet and Éric Soutif. Using a mixed integer programming tool for solving the 0–1 quadratic knapsack problem. INFORMS Journal on Computing, 16(2):188–197, 2004.